



A Comprehensive and Automated Approach to Intelligent Business Processes Execution Analysis

MALU CASTELLANOS
FABIO CASATI
UMESHWAR DAYAL
MING-CHIEN SHAN

castella@hpl.hp.com
casati@hpl.hp.com
dayal@hpl.hp.com
shan@hpl.hp.com

Hewlett-Packard, 1501 Page Mill Road, MS 1142, Palo Alto, CA, 94304, USA

Recommended by: Anoop Singhal

Abstract. Business process management tools have traditionally focused on supporting the modeling and automation of business processes, with the aim of enabling faster and more cost-effective process executions. As more and more processes become automated, customers become increasingly interested in managing process executions. Specifically, there is a desire for getting more visibility into process executions, to be able to quickly spot problems and areas for improvements. The idea is that, by being able to assess the process execution quality, it is possible to take actions to improve and optimize process execution, thereby leading to processes that have higher quality and lower costs. All this is possible today, but involves the execution of specialized data mining projects that typically last months, costs hundreds of thousands of dollars, and only provide a specialized, narrow solution whose applicability is often relatively short in time, due to the ever changing business and IT environments. Still, the need is such that companies undertake these efforts.

To address these needs, this paper presents a set of concepts and architectures that lay the foundation for providing users with intelligent analysis and predictions about business process executions. For example, the tools are able to provide users with information about why the quality of a process execution is low, what will be the outcome of a certain process, or how many processes will be started next week. This information is crucial to gain visibility into the processes, understand or foresee problems and areas of optimization, and quickly identify solutions. Intelligent analysis and predictions are achieved by applying data mining techniques to process execution data. In contrast to traditional approaches, where lengthy projects, considerable efforts, and specialized skills in both business processes and data mining are needed to achieve these objectives, we aim at automating the entire data mining process lifecycle, so that intelligent functionality can be provided by the system while requiring little or no user input. The ambitious end goal of the work presented in this paper is that of laying the foundation for a framework and tool that is capable of providing analysts with key intelligence information about process execution, affecting crucial IT and business decisions, almost literally at the click of a button.

Keywords: business process, business process intelligence, process analysis, prediction, metrics, star schema

1. Introduction and motivations

The research area of business process automation and management has been very active for over a decade now. A number of research projects were launched in this domain, and many papers have been published. Despite the strong initial interest, process automation techniques failed to gain acceptance in the market, primarily due to the high heterogeneity in the different process components to be integrated and to the lack of

middleware platforms (such as message brokers) capable of reducing or hiding such heterogeneity.

Recently, however, the availability of application integration solutions, the reduced heterogeneity caused by the standardization efforts (such as those taking place in the Web services arena), and the need of managing higher number of complex processes has generated both the opportunity and the need for supporting business processes by means of workflow management systems or other automation techniques.

These deployment efforts, as well as most of the research focus, have been centered on supporting the *modeling* of business processes and their automated *enactment*. The benefits of such automation are that processes can be executed faster, with lower costs (due to the reduced human involvement), and in a controlled way, since the enactment system can detect exceptions or delays in process executions and react to them in the way specified by the process designer.

However, as more and more processes become automated, the focus of both industry and academia shifts from deployment to process monitoring, analysis, and optimization. This is once again a matter of opportunity and need: the opportunity is provided by the fact that virtually every process automation platform logs execution data, which can then be accessed to provide users with information about such executions. The need is caused by the increased number of processes being automated and by the increased number of process instances being executed, difficult to manage without automated support.

Indeed, this push is demonstrated by the recent introduction of many tools and techniques that help users in gaining visibility over their processes (see, e.g., the efforts by Tibco or WebMethods) and by the increased number of publications on these topics [1, 12, 27].

Generally speaking, there are essentially three kinds of approaches to process execution monitoring and analysis (figure 1).

The first and simplest one consists in providing simple reports off the process execution database. Users can typically access reports showing basic process statistics, such as the average time taken to execute a step in a process or the status of each active process instance.

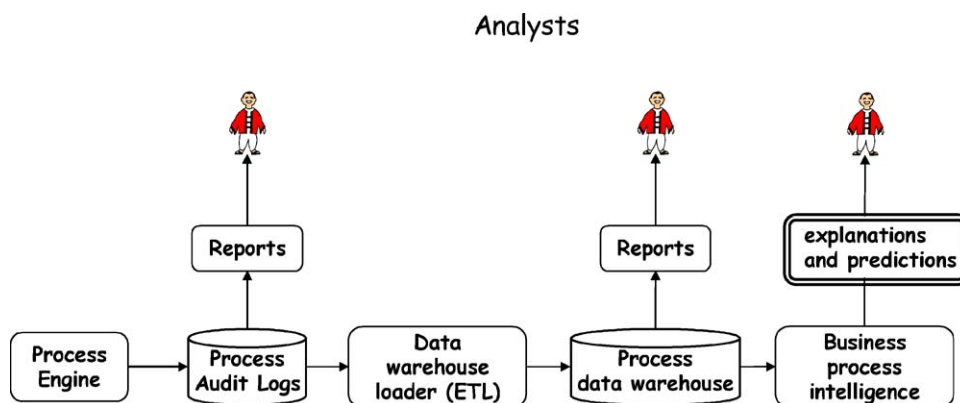


Figure 1. Different sophistication of the approaches to business process analysis and management.

This functionality is provided by virtually every system, including the first generation of workflow management systems appeared in the early nineties.

A more sophisticated approach consists in enabling OLAP-style analysis. In this technique, execution data are periodically collected into a data warehouse, and users are offered a number of different ways to look at the data, including in particular multidimensional analysis. Specific proposals in this category can differ considerably based on the sophistication of the data warehouse and on the capability of providing aggregated and summarized information that is easy to consume. Examples of reports that can easily be defined once a data warehouse is in place include the average number of exceptions occurred in a process depending on the resources involved in its execution, or the total execution cost by day. A few systems provide this kind of capability.

The third and most intriguing approach to process execution analysis involves the use of *business intelligence* techniques, to provide users with *explanations* of process behaviors of interest as well as *predictions* on running and even yet-to-be process executions. The adoption of business intelligence techniques causes a shift from the passive system behavior of the previous two approaches (where the user has to look for problems, “guess” their causes, and pose the right query to the system to verify whether the assumption was correct) to an active one, where it is up to the system to provide the user with information about problems and their causes. As an example, when business intelligence techniques are applied, users may be able to ask the process management system questions like “why are my processes slow?” and “how many instances will be started tomorrow?”. We refer to the application of business intelligence techniques to business processes as *Business Process Intelligence* (BPI).

Commercial systems do not currently provide BPI capabilities, and little progress has also been made by the research community (more details on these aspects are provided in the related work section, later in the paper). Currently, if intelligence analysis is required to address a specific problem (such as the one mentioned above about understanding why the cost is high), then users have to resort to long and costly ad-hoc projects involving considerable efforts and requiring data mining and engineering skills. Despite the considerable costs, the importance of sophisticated process analysis is such that companies do engage in these efforts to achieve the desired results.

This paper presents concepts, techniques, and an architecture that lays the foundations for business process analysis tools that can reduce the time and costs required for achieving the objectives described above by several orders of magnitude. Our goal is to be able to obtain explanations and predictions on a wide variety of process metrics and behaviors (such as the ones mentioned above) in an automated way, almost literally at the click of a button and with minimal or no user involvement.

The potential benefits of this achievement are very substantial, as results that normally would take a considerable time, skills, and effort can be instead achieved quickly and easily (although, as we will discuss in detail, sometimes at the lack of accuracy with respect to focused, ad-hoc projects). Not surprisingly, the challenges are substantial as well, and this is proven by the fact that no such tool exists on the market, despite the huge demand.

Our work towards achieving this goal is structured along several steps, that also correspond to what we believe our contributions in this paper are:

1. Identify classes of analysis and prediction problems in business process management that can be addressed with business intelligence techniques and whose solution can provide significant value to users.
2. For each class of problem, identify how the problem space can be abstracted and addressed by means of data mining techniques, and identify which techniques are applicable. In some cases this also involves the development of new mining algorithms or the modifications of existing algorithms.
3. Develop a way to automatically select significant process features to be included in the generation of the analysis and prediction models.
4. Develop a method and system that, based on the problem at hand, applies several mining algorithms and identifies the one algorithm, among the applicable ones, providing the most accurate results.
5. Process the results so that they can be used to provide information easy to consume both for human users as well as for automated system.

The initial results of our research effort have been made concrete and implemented within the *Business Process Cockpit* (BPC) [32]. The cockpit is a tool, developed at HP, that allows users to define and monitor business metrics on top of process execution data. When combined with the techniques presented in this paper, the cockpit evolves from a passive platform (in the sense described above) into a system that allows users not only to define metrics that they consider significant to measure their business, but also to have explanations about why process metrics have certain values (e.g., why the cost is high or the quality is low) and about the predicted value of such metrics, all at a click of a mouse and in just a few minutes.

Our presentation is structured as follows: we begin by introducing an example that will be used throughout the paper and that demonstrates the kind of problems we aim at addressing with our research (Section 2). In Section 3 we present the assumptions we make on the process model and present the concepts underlying the first generation of the Business Process Cockpit, the tool on top of which we developed our solution. Next, in Section 4, we discuss and classify different problems in business process management that can be addressed by means of data mining techniques, and show which technique is more applicable for each class of problems. Section 5 describes the approach to automated data mining in details, while Section 6 presents experimental results. Finally, Section 7 discusses related work and Section 8 outlines our future agenda.

2. A case study: Supply chain

This section introduces an example inspired by one of the projects we have been working on, and related to supply chain management (SCM). The example will be used throughout the paper to illustrate the concepts. The second part of this section presents a concrete example of analysis and prediction problems in this space, and emphasize the challenges that users need to face when addressing such problems. Our value proposition consists indeed in solving such challenges, not only for the supply chain example, but for any business process.

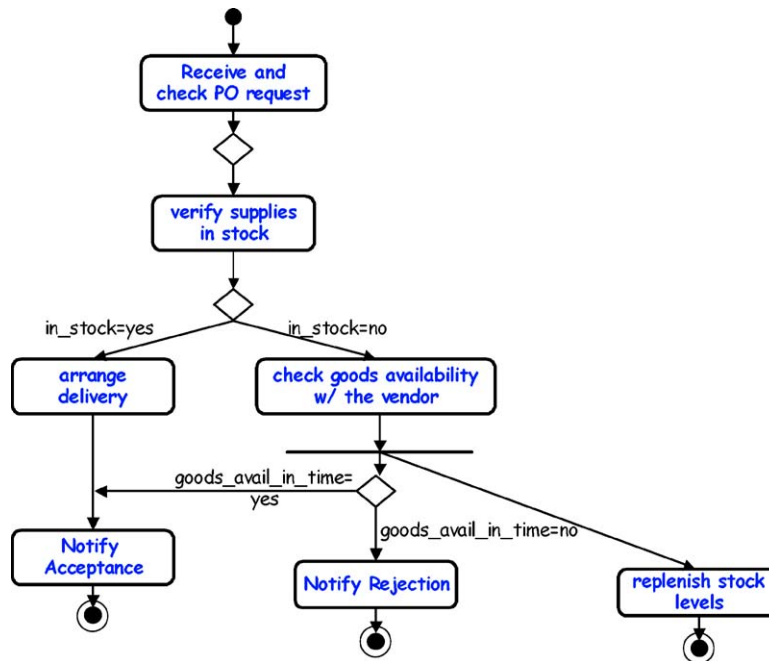


Figure 2. A process schema describing a supply chain business process.

2.1. The supply chain management process

The process described here is in fact simplified with respect to the actual one being implemented, as we have abstracted many of the details and removed the “technicalities” that are irrelevant to this paper. The process is depicted in figure 2, and its semantics is as follow: customers order goods electronically from a supplier, by sending a Purchase Order (PO). When a request is received, the supplier verifies whether it has sufficient amount of goods in stock. If so, it arranges for the delivery and notifies the customer. Otherwise, if the supplier’s warehouse does not have the required quantity of goods in stock, the supplier contacts the vendor to verify whether the vendor is capable of delivering the requested quantity of goods by the deadline requested by the customer. If so, the request is accepted, otherwise it is rejected. Regardless of the vendor’s ability to deliver goods in time, the supplier will order goods from the vendor to replenish its stock levels.

2.2. Analysis and prediction needs in the supply chain example

As mentioned in the introduction, it is important for the supplier to automate this process, as this enables reduced costs in operating the supply chain as well as improved efficiency. Once automation has been achieved, the focus of the supplier gradually shifts towards gaining

visibility into supply chain operations (to understand its effectiveness), and in particular towards identifying means to improve such operations. In fact, while automation provides crucial benefits, the ultimate goal for the supplier is that of being able to fulfill customers' requests. If automation does not contribute to this goal, then the supplier will have failed to meet its objectives, and most likely customers will bring their business elsewhere.

In this particular example, the supplier needs to maximize the number of orders that can be fulfilled, which means having goods in stock and establishing business relationships with vendors that can consistently deliver goods in the time allotted. All this requires the ability of:

- detecting problems quickly (e.g., identifying the inability to serve certain orders from certain customers on certain products). This is the most basic step, since it makes the supplier aware of the problems and can cause the initiation of further analysis or corrective actions.
- understanding what are the sources of the problem (e.g., is the supplier doing a poor job in maintaining adequate stock levels, or is the vendor often late in delivering goods, or are the customers' requests too demanding for the supplier's capabilities). This is important since by identifying the cause, the supplier can then take actions to remove such causes and therefore achieve more effective supply operations.
- predicting future load patterns (e.g., predicting the future order patterns, and in particular predict how many products will be ordered in a certain time window, to help adjust the stock levels accordingly).

All of the above are well known problems in the supply chain and warehouse management domain. Some companies address them with a simplistic approach (just monitor the stock level of a certain product, and order more when it goes below a threshold). Other companies use a more sophisticated approach, and employ data mining techniques to address the problem. The latter is clearly the most effective approach, and gives by far the best results, both in terms of analysis and predictions. However, it is clearly more complex and more expensive. In fact, consider the effort required in undertaking such an endeavor. In order to build such a sophisticated analysis and prediction tool, the supplier would have to:

1. Hire engineers and personnel skilled in data mining
2. Purchase data mining applications (such as SAS)
3. Start a data mining project, which involves several steps, such as
 - Identifying which data mining techniques can be adopted to solve each problem (or, in other words, mapping each problem to be solved to a data mining problem)
 - Collecting data into a data warehouse, to facilitate subsequent processing
 - Identifying which data to use and assemble the necessary data structures
 - Analyzing the quality and cleanliness of the data
 - Identifying the features that are more highly correlated with the problem at hand
 - Tuning the many parameters that characterize the data mining algorithm
 - Executing the algorithm

- Evaluating the performance (may imply to go into a loop to improve it)
- Presenting the results to the users in a manner that is easy to consume.

As the reader can appreciate, the above steps are both complex and expensive in terms of time and money. It is not uncommon for such projects to last for several months and be characterized by 7-digits cost figures. Furthermore, if the environmental condition changes (from all sorts of reasons, ranging from sudden economic recession to modifications in the competitive landscape) the whole analysis process has to be repeated, or at least fine-tuned. Despite all these drawbacks, the results of this effort are so important that many companies (especially large suppliers) are actually willing to undergo this sort of complex and expensive projects.

Imagine now how beneficial would be for customers to have all the benefits that the application of data mining techniques can provide readily available to them (almost literally at the click of a button), at no cost at all, and without the need of starting lengthy projects or acquiring sophisticated skills. And imagine how more beneficial this would be if it was not only applicable to a specific problem or a specific business process, but it could be available for any process and for virtually any kind of business problem that needs to be solved. The implications for the users would be enormous. This goal of this paper is indeed that of laying the foundations to achieve this ambitious objective.

3. Assumptions on the business process model and on the executing IT infrastructure

Since in the field of workflow management there is still some confusion in the way different terms (such as “process” or “activity”) are used, in this section we introduce the terminology that we adopt and we describe the basic assumptions that we make on the process model. The final part of the section describes the process analysis platform, called Business Process Cockpit, that constituted the starting point for our BPI work.

3.1. Terminology and assumptions on the process model

We use the term *business process* to denote a set of activities that collectively achieve a certain business goal. Examples of processes are the hiring of a new employee or the processing of an order. The term “business” is used to distinguish it from other kind of processes, e.g., operating system processes. A *process schema* (or *workflow schema*) is a formal description of a process, suitable to be executed by an automated system, called *process engine* (or *workflow engine*). An individual execution of a process schema is called *process instance* (or *workflow instance*). An invocation of a service constitutes a *service instance*. A process schema is defined by the following (see also the example of figure 2):

- A set of *activities* that are part of the process schema. For example, the collaborative supply chain process of figure 2 includes activities such as *submit for approval*, *notify acceptance*, or *notify rejection*. Activities are denoted by rounded boxes in the figure.

- A set of directed *arcs*, that connect a source activity S with a target activity T, with the meaning that activity T is to be started as activity S ends. Arcs can be associated to transition conditions, evaluated at the time the source activity terminates and controlling whether activity T should indeed be activated when S ends. Activity T is started upon completion of S only if the transition condition is true. If the arc condition is not specified, then it defaults to true.
- A set of *routing nodes*, that are decision points defining how the process execution should proceed. The figure shows two kinds of routing nodes: *diamonds*, denoting conditional execution (only one of the activities connected in output is activated, based on the arc's transition conditions), and *horizontal bars*, denoting parallel execution (all activities connected in output are activated). Furthermore, the diagram includes *start nodes* (denoted by circles) and *completion nodes* (denoted by a double circle). Start nodes denote entry points in the flow: when a new instance is started, the activity that is connected in output to the start node is activated. End nodes denote that the process instance execution (along the path where the end node is placed) should not proceed further. This notation has been borrowed from that of UML activity diagrams.
- A set of data items (also called *variables*), describing data local to each process. For example, the CSC process schema includes variables such as customer name, requested product, and expected delivery date. Variables can be set at process instantiation time, and can be modified by activity executions. Their purpose in a process schema is to transfer data among activities and to enable conditional executions, since variables are referred to within transition conditions.

The definition of an activity typically includes the specification of:

- The *service* to be invoked by the activity. For example, activities *notify acceptance*, or *notify rejection* both invokes service “send email”, provided by an email web service.
- The resource that should execute the activity. For example, the URI of the send email web service to be invoked. Note that the resource can be specified statically or determined dynamically, based on the value of a variable.
- The process data items to be passed to the resource upon invocation and received from the resource upon completion of the work.

Process models are much more complex than what described above, and process schemas typically include additional details (e.g., exception handling or transactional behaviors). However, the simple description given above is sufficient for the purposes of this paper. We further observe that these assumptions are very simple, and are fulfilled by virtually any process model (although each has its own minor variations). As such, the research described in this paper is applicable to virtually any process management system.

We further assume that process executions are logged in a database. This database includes information on *process instances* (e.g., activation and completion timestamps, current execution state, name of the user that started the process instance), *service instances* (e.g., activation and completion timestamps of a service invocation, current execution state, name of the resource that executed the service, name of the activity in the context of which the

service was executed), and *data modifications* (e.g., the new value for each data item every time it is modified).

Again, these assumptions are met by virtually any system as described in the introduction, and some systems even provide a data warehouse where process execution data are cleaned and stored in a manner that facilitates their analysis. While our algorithms have been developed based on a specific process log database schema, they can be easily adapted to other schemas since the concepts are the same and are equally applicable. In particular, they can be applied to Web services and their composition, where the problem is in fact simplified due to the homogeneity of the components and the increased level of standardizations in the composition models and languages [3].

3.2. The Business Process Cockpit

Our approach has been developed on top of (and integrated into) the *Business Process Cockpit*, a tool developed at HP Labs that facilitates the definition and computation of metrics on top of business process execution data. To make the paper self-contained, in this section we briefly present the functionality of the cockpit and its overall architecture, depicted in figure 3. A more detailed description of the cockpit can be found in [6, 32].

The cockpit performs three main functions: The first is that of collecting process execution data into a data warehouse, whose schema is shown in figure 4 (many details are omitted for clarity). This involves extracting data from the process logs, cleaning the data, and transforming it into a multidimensional format (implemented by means of a star schema

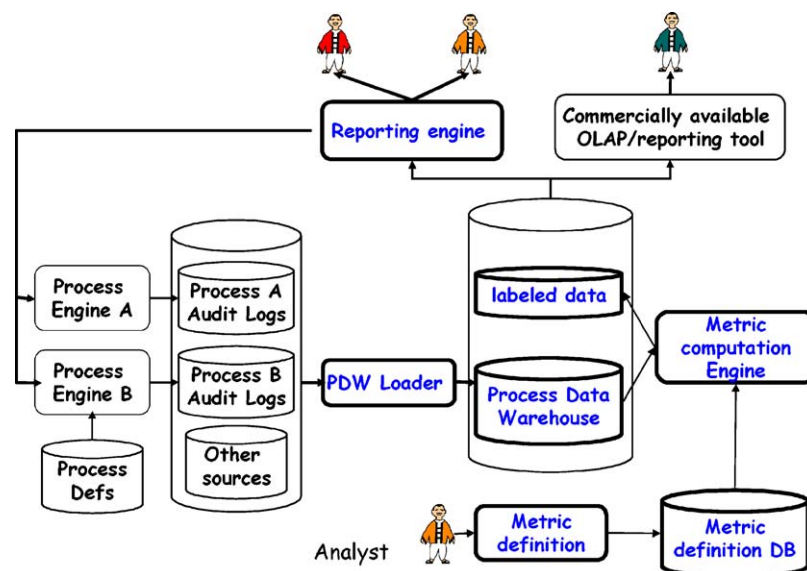


Figure 3. Architecture of the Business Process Cockpit (cockpit components shown with a thicker border).

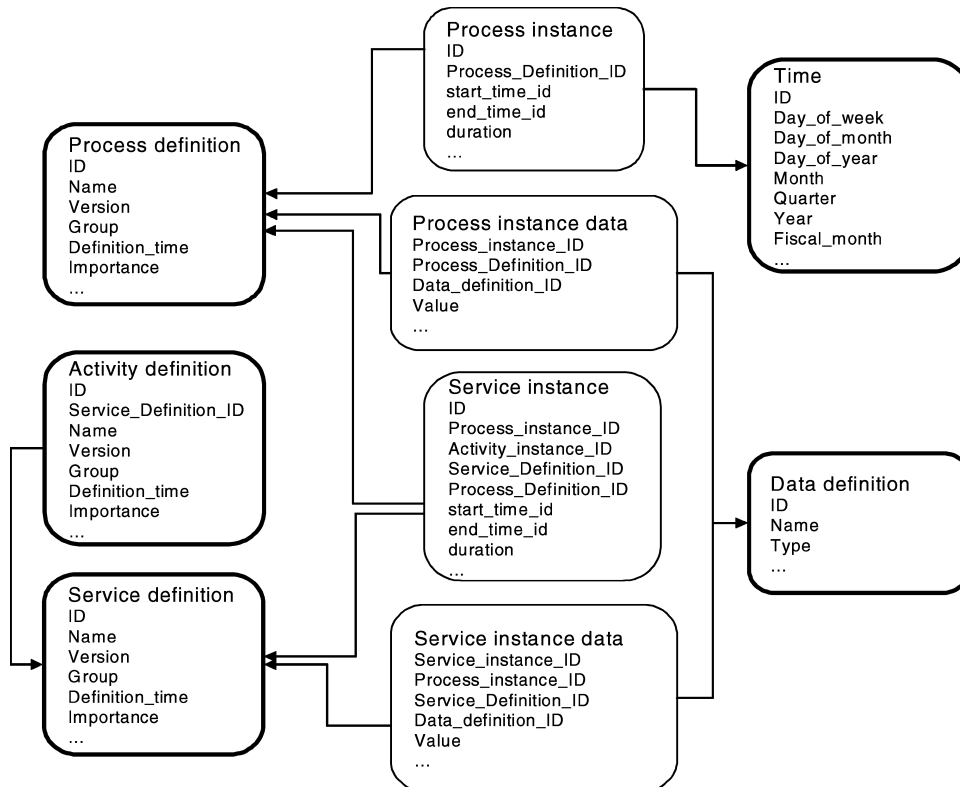


Figure 4. Star schema of the process data warehouse (overview). Dimensions are shown with a thicker border. Arrows denote foreign key relationships (only the ones between facts and dimensions are shown).

on a relational DBMS), which supports better analysis through easier query definition and faster query execution.

This function is performed by the PDW Loader (PDW stands for Process Data Warehouse), that is executed periodically, according to user-defined configurations, and in a way that minimizes the impact on the operational system (the process engine). Data in the warehouse can be accessed by means of any reporting tool, such as Crystal Reports or Brio. The part of PDW loader that interfaces with the audit logs is dependent on the specific format of the logs themselves, as each workflow vendor supports a different model and has a different format (database schema) for the audit log. However, although the details are different, the basic concepts are the same, and is therefore possible to adopt the same methodology in terms of how data are extracted from the logs, which mechanisms are used to minimize the impact on the workflow engine, and the like. Details on techniques for extracting data from process logs are provided in [6]. The part of the PDW loader that interfaces with the warehouse is instead generally applicable, as the PDW schema itself is meant to be applicable to a large variety of process models and tools. Indeed, one of the contributions of our work

has been the identification of a PDW schema that, based on the typical workflow reporting needs, enables easy query specification and fast query execution.

The second function provided by the cockpit consists in the ability of computing user defined metrics on top of process data. For example, analysts can define a notion of “cost” or “quality”, and then define how these can be computed (typically by means of an SQL query) based on process execution data. These definitions, specified through the cockpit definer, are saved in a database and accessed by the metric computation engine that, as new data is loaded into the warehouse, determines the metric values. In essence, the metric computation engine “labels” processes, process instances, and other entities in the warehouse with the computed values of the user-defined metrics. The results are also stored in the warehouse, so that users have can analyze metric data, again by means of a reporting tool. For example users will be able to see reports such as “average cost by process” or “process execution quality by week”.

Finally, the cockpit includes a reporting tool that allows users to view report data. In contrast to a general-purpose reporting tool, the cockpit reporting engine is specialized to show business process data, and makes it easier to navigate through the data contained in the warehouse and define reports on the fly.

In summary, with the cockpit, users can define metrics with a point-and-click GUI and have these metrics computed automatically. However, this cockpit is still “dumb”, meaning that it “just” reports on what the user-defined metrics, but it provides neither explanations on why a metrics has a certain value, nor predictions on the future value of the metrics. These limitations are addressed by building “intelligence” into the cockpit. This aspect is described in the remainder of the paper.

4. Data mining and a taxonomy of analysis and prediction problems

As stated earlier, the goal of the BPI Cockpit is to provide the necessary functionality to automatically obtain explanations and predictions on a wide variety of process metrics and behaviors. The underlying technology is data mining. However, data mining techniques are difficult to use, not only data needs to be prepared, but users needs to be experts in data mining to decide which techniques to use, tune the algorithms and interpret the results. Moreover, the effort required to mine business processes using existing off-the-shelf components is pretty big and generally turns into a long and costly project to cover the whole data mining cycle. The idea behind BPI Cockpit is to offer out-of-the-box data mining capabilities that take care of all the tasks involved in the knowledge discovery cycle of business processes so that process execution data can be mined with minimal or no user effort and without requiring data mining experience.

The first step towards this goal was to analyze the classes of business process analysis and prediction problems whose solution provides value to the users. We do not claim that this taxonomy is complete, but it reflects the needs that have been expressed by the businesses. We use four dimensions to partition the problem space into a taxonomy of problem classes, as depicted in figure 5. These dimensions are *task*, *scope*, *focus* and *status*. Besides helping the reader to conceptually understand the different kinds of problems, this classification is also useful in that it identifies different approaches that can be taken to mine the process

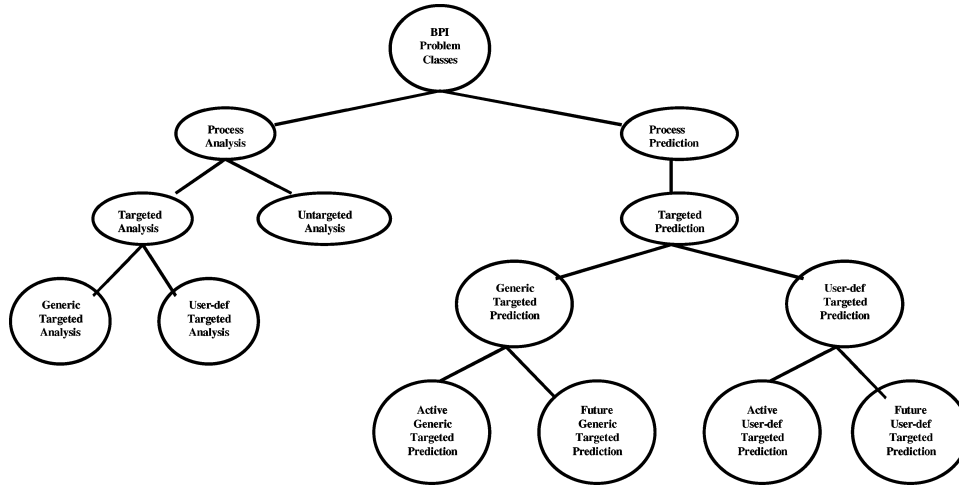


Figure 5. Taxonomy of predictive mining problems for business processes.

data, and specifically to select the features. In this section we analyze each dimension, and discuss which data mining techniques can be used to address the problems in each one. In order to make this paper self-contained, we start with a brief description of relevant data mining concepts.

4.1. Data mining concepts

Data mining [16] is defined as the process of discovering patterns in substantial quantities of data. The process must be automatic or (more usually) semi-automatic and lead to the discovery of useful patterns that allow making non trivial predictions on new data. There are two extremes for the expression of a pattern: as a black box whose innards are effectively incomprehensible and as a transparent box whose construction reveals the structure of the pattern. Both, make good predictions, the difference is whether or not the mined patterns are represented in terms of a structure that can be examined, reasoned about, and used to inform future decisions. Those patterns are usually called *structural* patterns because they capture the decision structure in an explicit way. In other words, they help to explain something about the data. Many data mining techniques look for structural descriptions of what is learned, descriptions that can become fairly complex and are typically expressed as sets of rules or decision trees. Because they can be understood by people, these descriptions serve to *explain* what has been learned, i.e. to explain past behavior as well as to explain the basis for new predictions. The black-boxed patterns, even though they do not support any reasoning about them, are good enough in themselves by providing predictions that in some cases could even be superior in performance. A popular data mining techniques that does not look for structural patterns but that often outperforms those looking for them is support vector machines [9].

So far, we have been talking about prediction but we haven't given a definition of the prediction problem yet. *Prediction* can be viewed as the construction and use of a model to assess the class of an unlabeled sample, or to assess the value or value ranges of an attribute that a given sample is likely to have. In this view, classification and regression are the two major types of prediction problems, where *classification* [16] is used to predict discrete or nominal values, e.g., the performance of a process instance as "fast" or "slow", while *regression* [16] is used to predict continuous or ordered values, e.g., the duration of a process instance. Decision trees [16] are a typical technique used for classification, while support vector machines [9] can be used for both, classification and regression. Time series [5] is a specialized type of regression where measurements are taken at fixed intervals over time for the same variables (features), e.g. the average cost of daily orders and the goal is to predict the next value in the time series. Occasionally, when the goal of a time series can be stated in terms of a category or Boolean value, classification is more appropriate and may be easier to solve. For instance, predicting whether 90% of the process instances of next day will violate a service level agreement (SLA) on their duration instead of forecasting the number of processes that will violate the SLA.

In predictive data mining, samples of past experience with known answers, i.e. labels, (notice our abuse of the term "label" to imply both, discrete and continuous values, even though strictly speaking the term refers to discrete ones) are examined and generalized to future cases, i.e. the mission is to learn decision criteria (i.e. patterns) for assigning labels to new unlabeled cases. The labeled examples used to learn a pattern constitute what is called a *training set*. Data mining algorithms that use training sets belong to the category of supervised learning which is a form of learning by examples. Decision trees, decision rules and support vector machines belong to this category. On the other hand, there are unsupervised techniques where learning is by observation and therefore do not rely on predefined classes and training sets. Clustering [10] is the typical technique in this category and consists of grouping the data into classes or clusters so that objects within a cluster have high similarity in comparison to one another, but are very dissimilar to objects in other clusters. Given the above notions, we can now proceed to describe our taxonomical dimensions.

4.2. Task

There are essentially three classes of problems that can be addressed using business intelligence techniques: process definition discovery or improvements, process analysis, and predictions.

Process definition discovery refers to the problem of identifying a process definition (schema) from IT execution data. For example, assume that a company is performing a set of activities, supported by an IT infrastructure (but not necessarily by a process engine). A typical scenario is that different systems execute the different steps in the process, but the coordination among the steps (the business process logic) is either done manually or implemented by means of a third generation programming language (e.g., Java). If the execution (and in particular the interaction among the different systems) is logged, then it is conceptually possible to derive a skeleton of a process definition by looking at these data,

typically by discovering ordering relationships and correlations among events. Discovering the process definition can be useful to help analysts understand how the coordination among systems is actually achieved. In a way, this can be seen as a form of assisted bottom-up process design, where the process definition (or at least part of it) is derived from the actual implementation. The discovered definition can be then used to support the process with a process engine [1]. Even though process discovery is a conceptually interesting problem, BPC does not currently address it because the initial research we have done is not yet at the adequate level of maturity to provide practical useful results.

Process analysis refers to the problem of detecting “interesting” behaviors of one or more process executions and of providing explanations for the situations in which such behaviors typically occur. For example, a classical problem is that of identifying process instances that have a “low performance”, a “high cost”, or that fail to meet certain quality criteria, and provide explanations about the situations in which such performance degradations take place. Because the key term here is *explanation*, the appropriate data mining techniques to apply are those that discover structural patterns (see Section 4.1), i.e. decision trees and decision rules. Specifically, the idea here is to identify the characteristics or features of a process that contribute to determine the value of the performance metric and classification patterns on the values of those characteristics expressed as a decision tree or decision rules.

By presenting the user with a graphical representation of a decision tree or a natural language version of the rules produced by the mining algorithms, the user can understand the characteristics of the process instances for which certain metric values occur, analyze their root cause and make more informed decisions. For instance, the tree may indicate that when a given supply chain process starts during the last ten days of the month and the product ordered is a laptop model HP 5400, the order is usually delayed. This is important information that can help the business manager understand the reasons of the delay and take some action to decrease the number of SLA violations on the duration of the process [7]. The limitation of these techniques is due to the fact that they do classification, and therefore they can only be applied to problems with categorical or discrete labels. When labels are continuous, the same kind of tree representation can be used, but the leaf nodes of the tree would contain a numeric value which is the average of all the training set values that the leaf applies to. This kind of tree is called regression tree [4]. Even though regression trees are often large and complex for being interpreted, they constitute an alternative to obtain explanations when nothing else works better. However, for the analysis of metrics, it is often possible to map a regression problem into a classification problem by partitioning the metric values into significant segments or classes. For example, instead of learning a regression tree for *order_cost*, the cost values could be partitioned into the classes “high”, “medium” and “low”, then, classification techniques and in particular, those that learn structural models, can be used to provide explanations for these classes. In general, we want to apply decision trees or decision rules whenever possible to obtain descriptive models that can be interpreted by the end user.

Process prediction refers to the problem of providing analysts with information about the outcome of a running process instance, or even information about instances yet to be started, such as how many orders will be placed tomorrow or what will be the predicted total order value. As explained in Section 4.1, both classes of predictive data mining techniques,

classification and regression, can be applied for prediction. It all depends on the type of the value to be predicted: When the value is categorical, either prediction technique can be applied, whereas when it is continuous only regression is applicable. For example, to predict the performance of a process instance as a categorical value “good” or “bad”, either decision trees, decision rules or support vector machines (this last one in classification mode) could be used. In contrast, to predict the performance as a numerical value ranging for instance from 0 to 100, we would use support vector machines (this time in regression mode). Observe that the nature of the learned pattern (whether it is structural or not) is irrelevant for prediction. In fact, in contrast to process analysis, the learned model is not used to provide explanations. The accuracy of the prediction is all that matters and therefore, as we will see in Section 5, when several predictive techniques are appropriate, the extended functionality of the cockpit applies all of them to learn models from labeled historical data (training set) and chooses the model with the best performance, regardless of which model is more amenable to be interpreted by a human user. The preferred technique to be used also depends on whether the prediction is all that is needed, or whether users also want to have an explanation of why a certain prediction has been made. We will come back to this issue later in the paper, when discussing how BPC handles process predictions. However, we observe here that the key point is that users need to have predictions as early as possible in the process, possibly with limited accuracy. Then, as the process execution proceeds and more data become available, predictions should become more accurate. Therefore, each process prediction technique should be able to factor in the fact that new data on a process instance become continuously available (more features become known). Note that this is *always* true and we always get new information about the process every time we look at it, since even if the process is suspended and nothing is happening, the fact that the process has been suspended for a certain time may prove to be a useful predictor for a certain metric. In all this, as stressed before, the key point is that we need to automate the procedure of deriving and applying these models. We will show in Section 5 how this is done in BPC.

4.3. Scope

There are essentially two kinds of metrics that can be analyzed or predicted: generic and user-defined. *Generic* metrics are applicable to any process, and are needed in many analysis scenarios. For example, each process has a notion of duration, and analyzing or predicting the duration is a common need. Other examples involve analyzing why a loop is executed more than a given number of times, predicting how many times the loop will be executed, or identifying when a process instance takes a given route (e.g., an order request goes down an accept path or a reject path). For the particular case of SLAs which are in fact Boolean metrics, a classification of generic SLAs is given in [7].

On the other hand, *user-defined* metrics can relate aspects that are specific to a particular process or to a group of processes, and cannot be (easily) generalized. For example, users may want to analyze or predict the cost of processing an order, where the cost is specified as some function over process instance execution data.

From a BPI perspective, the difference is that generic metrics are known in advance, and therefore it is possible to experiment with them to build more accurate ad-hoc models

and techniques for analyzing and predicting them. More specifically, in this case we (as developer of the BPI solution) are aware of not only the semantics of business processes in general, but also of the semantics of the metric being analyzed, and therefore this knowledge can be embedded in the cockpit to better tailor the algorithms and identify features that are typically correlated to the metric value. Observe, however, that even if the target metric is known and the database schema for process execution data is known, there may still be the need for the user to be involved in the loop to build explanation models given that the classification patterns might result obvious to him (automatic feature selection cannot anticipate this since it depends on what the user considers interesting). In such case, a GUI lets him easily mark those attributes in the patterns that are not interesting so that the algorithms do not use them anymore in building the classification models, this is what we call semantic feature selection. Note that this kind of feature selection is only necessary for explanation models and not for prediction because the patterns in those models have to be interesting and interestingness is a subjective issue. However, as we will see later, the rest of the work involved in the knowledge discovery process, i.e. preparing process execution data to assemble training instances, selecting process features according to their influence in determining metric values, selecting adequate data mining algorithms, tuning them, comparing and validating them and creating visual representation of the models when appropriate, is done automatically as we will show in Section 5.

In contrast, user-defined metrics are harder to support, since BPI is not aware of their semantics. However, the knowledge of the user-defined functions for computing these metrics can be helpful in identifying a strategy for building the model. Nevertheless, as we will see in Section 5, there is some added complexity in various steps of the data mining process lifecycle for this kind of metrics.

As can be appreciated, this dimension does not have any influence on the kind of technique to be applied to build analysis and prediction models. However, having knowledge about the semantics of a metric in advance is useful to predefine specific solutions (not the models themselves) for it. These solutions are derived either empirically, by gaining experience on manually applying the data mining algorithm to solve a specific problem over several cases, or a priori, thanks to the knowledge of the semantics of a metric (and of business processes in general). This involves in particular the selection of features to be used as input to the mining algorithm, to be selected from a potentially unbounded space, as discussed later. Examples of empirical learning for specific metrics will be discussed later in the paper.

In the case of user defined metrics, since there are no predefined solutions, there are occasions when it is necessary to prompt the user for some information in some steps of the overall process.

4.4. Focus

Analysis (also called explanation in this paper) on business process execution can be targeted or untargeted. In *targeted* analysis, we ask the BPI system to explain why a process metric has a certain value. For example, we want to know why the duration is above a certain threshold, or why the cost is “high”. In other words, targeted analysis assumes that users already know what they want to analyze. As stated previously, analysis is performed by

techniques that generate structural models, i.e. decision trees and decision rules. On the other hand, prediction is always targeted because the user always knows what he wants to forecast, i.e. the value of a metric at a future time. Both, classification and regression techniques are applicable here (e.g., decision trees, decision rules, and support vector machines), and the decision about which one is applicable depends on the other dimensions discussed in this section. However, analysis is not always targeted. In some cases, users are interested in finding “interesting” patterns in the data that may be indicative of some characteristics of the processes that they were not aware of, or some “oddities” that can signal problems or opportunities for improvements. For example, it would be very helpful for users if the system could identify and point out process instance executions that fall outside normal execution patterns, such as orders taking an unusually short time to be fulfilled, or orders for a particularly high or low value.

Indeed, while analysts can explicitly specify a few cases of interest, the reality is always more complex than the imagination, and it is unrealistic to think that the analysts is able to define all the different possible interesting issues. Untargeted analysis therefore comes to help in that it does not require the definition of what to look for, but autonomously identify executions that fall outside the normal pattern. Statistical techniques, ranging from simple quartile-based techniques used for outlier detection to more sophisticated ones used for quality control such as control charts, can be used. Another example of untargeted analysis is clustering, where different process instances, services, or resources are grouped together according to some similarity criteria. Untargeted clustering can be helpful to unveil unexpected groups of processes that share certain relationships between their features which the user had no idea existed. Thus, it provides new insights into the processes and their executions. For example, the result of clustering can identify a cluster of process instances that use certain resources for the execution of certain activities on certain days when the amount being processed is ten thousand dollars. This discovery can trigger an investigation of those process instances, possibly finding them as fraudulent. In the remainder of this paper we are concerned with targeted analysis, since this is what appears to be the most pressing requirement.

4.5. *Status*

An important distinction in the kind of prediction problem to be solved is between making predictions about the future behavior or outcome of an instance that is already running versus making predictions about instances that are still to be started.

Predictions about active instances are useful for various purposes: for example, they may indicate the likely outcome of the process execution (i.e., whether an order will be accepted or rejected), the expected completion time, and in general they can estimate the value of any user-defined metric. A specific subset of predictions that is of particular interest is related to whether a certain exception (i.e. abnormal behavior of a metric) is likely to occur in a running process instance (SLAs are a particular class of exceptions). This information is crucial, since it can be used to try to prevent the exception (or SLA violation) either by feeding the information to an automated system, or at least by notifying a system manager who can then take appropriate actions, and to reduce the impact of the exception. For

example, an interesting problem involves predicting whether a process instance will violate an SLA on its duration, i.e., not meet its deadline (e.g., whether goods will be delivered late or not). If the prediction for an instance is that it will not complete in time, then a specified user can be informed, so that he can try to correct the problem. In practical scenarios, this involves making telephone calls to the supplier asking to speed up the delivery, or going for air rather than ground shipping. Note that these actions could also be encoded as part of the process model and, therefore, reactions to a predicted failure could be automated. However, even if it is not possible to react to an exception in order to avoid it, exception prediction information can still be used to take actions that reduce the damage caused by the exception. For example, knowing that an order will be late is helpful in that the customers can be informed about the late delivery, and can plan accordingly.

Process prediction can also be used to optimize system performances. For example, if the algorithms predict that a certain resource will be needed, the system can then reserve such resource. This is particularly useful when processes are run on top of a utility infrastructure such as the Grid.

A separate class of prediction problems is about future instances, i.e. instances that do not exist yet. This class of prediction problem involves trying to identify if instances of a certain process will be activated, when, in what volume, and what could be their behavior and outcome. For example, a typical problem involves predicting the volume of request for quotes for the following day or week, which equals to predicting how many instances of a given process will be started. An even more difficult problem involves predicting how many of these requests yet to be received will actually be approved and results in actual orders.

From a data mining perspective, the two approaches require different techniques. In the first case, since a process is running, data about the (partial) execution is available. This data is likely to be correlated with the metric to be predicted. The more the process execution advances, the more data becomes available about the execution, and the more accurate is the prediction. For example, the prediction that can be made about the process duration and outcome will be more and more accurate as the process nears its completion. This means that it is essential for the prediction algorithm to take advantage of the partial execution data, and be able to provide more and more accurate predictions as the execution proceeds.

The approach we follow to predict metrics on running processes is to use classification and/or regression techniques that, as explained above, are appropriate in this context (classification is for categorical labels whereas regression is for continuous ones). For example, by analyzing data on completed executions, the classifier may find that the customer initiating the process and the type of product ordered X have a strong correlation with the process duration and the deadline expiration. Therefore, based on the obtained classifier, these two variables (whose value becomes known as the process instance is started) can be used to predict deadline expirations.

A problem to be addressed here is that in many cases the classifier is likely to be based on variables that are not available as the process instance starts, but that instead become defined during process execution. For example, a predictor for order rejection could be based on the vendor with whom the supplier checks goods availability when he finds that there are not enough supplies in his stock, which is not known at process instantiation time. Therefore,

this classifier would be useless for making any prediction until the time that node `verify supplies in stock` is invoked and the vendor becomes known.

To address this problem, our approach automatically identifies the different execution stages of processes (a stage is characterized by a final node and the path followed to get to that node), creating the corresponding training sets from completed instances (but using only execution data that was produced up to the given execution stage) and building several classifiers (or regression models), one for each stage in the process execution, that makes use only of features that are available at that stage and that is therefore applicable for making predictions when the process instance is in that stage. For example, a classifier could be based only on process instance features known at process instantiation time (e.g., activation time and input parameters), and be therefore applicable for generating prediction as a new process instance is created. Another classifier could be created based on features available up to the execution of an activity A, and only include as features those process characteristics on a given path that are defined up to that point (e.g., duration, executing resource, and results of each activity executed before A, as well as of A itself). This model can be used to make predictions after activity A has been completed. In this way, it is possible to use classifiers generated for the early stages to make predictions early in the instance execution (typically with limited accuracy), while classifiers for the later stages will have higher accuracy. An example of this is shown in Section 6.

The case of predictions on future instances is quite different in nature. The typical problem here is the prediction of the value of a metric that has been aggregated by time. For example, the analyst has created reports such as the daily cost of processing orders or the total number of orders processed per week, and they want to have predictions for the same metrics at some future day or hour. Time is a crucial element here since all we have is the time for the prediction and the time of previous measurements, which constitute a *time series* [5]. When a metric is aggregated by time, its values correspond to measurements taken at regular time intervals. In the example above, the daily values of the cost of processing orders and the weekly number of orders processed. While the typical goal is to predict the next unit in time, a period that corresponds to the immediate future, a goal can be any number of units in the future. For example, the user can request the prediction for the total number of processing orders next week, but he could also request the value for three weeks after that. However, the farther out in the future, the more difficult and less reliable the forecast is. Whatever this period is, the time series is transformed into a case format where the real challenge is the specification of the window or time lag.¹ For example, if the prediction is for the next day, how many previous days do we have to include in each case? Different values for the window size have to be tried and the one that gives the best performance has to be identified. Further challenges are whether to compose new features from time series given that sometimes some kind of aggregate, like average, or some function of the most recent value like ratio or current difference, are better predictors. An example of this is given in Section 5.1.1. Seasonal adjustments are also challenging and for many time series are essential for forecasting accuracy. As stated in Section 4.1, time series is a regression problem and occasionally can be approached as a classification one.

Figure 5 describes the taxonomy of analysis and prediction problems on business process metrics that we have identified and for which we present a business process intelligence

Table 1. Data mining techniques for the problem classes.

	Targeted						Untargeted
	Process prediction						
	Process analysis		Active instance		Future instance		
	Nominal metric	Numeric metric	Nominal metric	Numeric metric	Categoric metric	Numeric metric	
Decision tree	X		X				
Rule model	X		X				
SVM			X	X		X	
Regression tree				X			
Clustering							X

solution. Table 1 summarizes instead the data mining techniques that the current implementation of the cockpit applies to solve each class of problems in the taxonomy. Our intention is not to exhaustively cover all the possible techniques that are applicable to each class, just some that because of their popularity, intuitive interpretation, or superior performance have been chosen. Future versions of the BPI engine will include other techniques to extend its functionality according to new requirements and new advances in data mining.

5. The business process intelligence engine

The previous sections have provided an overview of the goals of our work and of our motivations, along with a discussion of the problem space. We have in particular stressed that performing intelligent process analysis and prediction is a crucial need for many companies, but implementing such functions is a costly, lengthy, and ad hoc activity. In the following we present the architecture of a system, of which we have implemented a working prototype at Hewlett-Packard, that automates the entire mining process for performing analysis and prediction on business process metrics, making them available almost literally at the click of a button. The core of the functionality is embodied into a newly developed component of the cockpit, called *Business Process Intelligence Engine*, and described in the following.

5.1. Architecture

The BPI engine has been implemented as an extension to the BPC. Figure 6 illustrates the extended architecture of the BPC. The figure shows that data from the Process Data Store is read by the BPI engine in order to be mined and predicted upon. These data are not only process execution data along with metric measurements used as labels, but it also comprises metadata about the entities, attributes, and metric definitions. The output (explanations and prediction models as well as the predictions themselves) is written to the Process Data Store

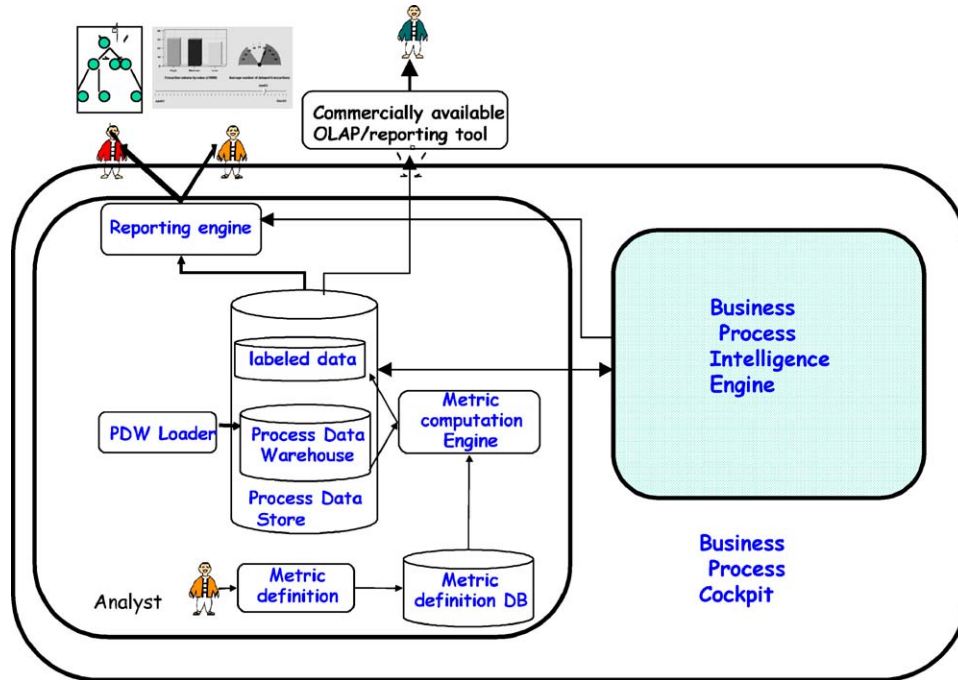


Figure 6. Extended Architecture of the Business Process Cockpit.

and is made readily available to the business analysts through either the Report Engine or a third party reporting tool.

The BPI engine embodies all the functionalities necessary to cover all the steps in the data mining process. The main distinction is that these functionalities have been specifically tailored for business processes. Thence, most of the effort that a user would normally have to undertake to mine business processes has already been done when building the engine so that the user can get out-of-the-box explanations and predictions. The architecture of the BPI engine is depicted in figure 7. For presentation purposes, we have grouped the modules of the BPI engine in two categories: *preparation* modules and *data mining* modules. The presentation order of the modules obeys to the flow that is typically followed during the data mining process. Scrubbing of process instance execution data is not discussed here because it is a task of the PDW Loader (see figure 3) which is described in [6].

5.1.1. Preparation modules. Data preparation is a critical activity in any data mining application. The key problem here is that we are looking at automating the preparation process, possibly with minimal or no input from the user. In the general case, that is, in arbitrary domains, automating this is impossible as the feature space is too large, possibly even unbounded. Even without considering the problem of feature selection, we would anyway be faced with the issue of accessing different kinds of data in different formats,

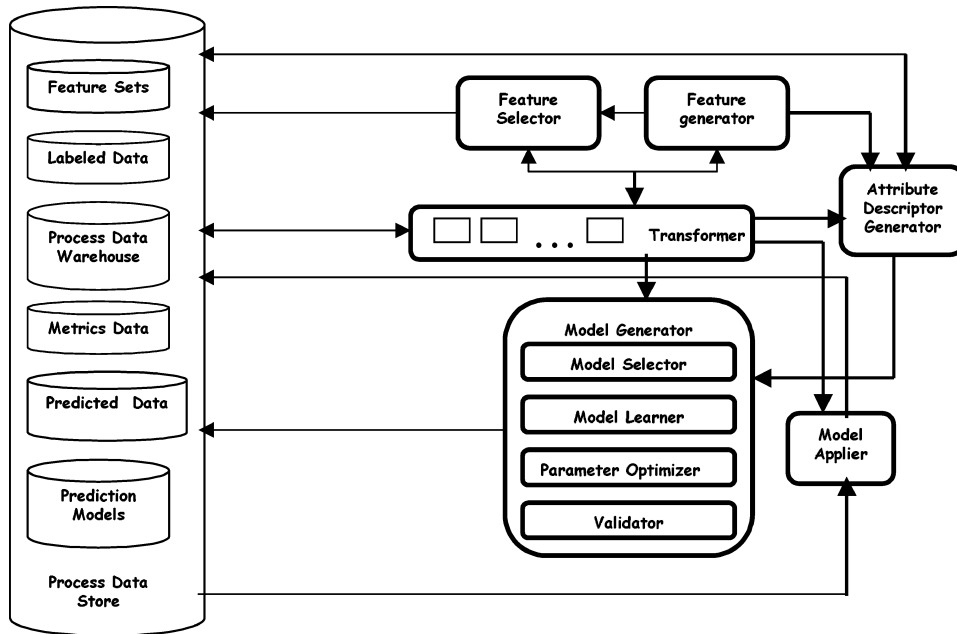


Figure 7. Simplified architecture of the business process intelligence engine.

so that it does not really make sense to think of a generic turn-key application that can act as a universal data collector and feature selector. However, it is our contention that this is possible in business processes. There are several reasons:

1. All processes supported by a given process management system have the same data model. Even if different engines are used, the PDW will take care of resolving heterogeneity and providing a fixed data model.
2. Although, even in this fixed data model, the number of possible features to be selected is in general unbounded, experience proves that there are certain features (original and derived) that are more highly correlated than others with most process metrics.
3. For generic metrics the problem is farther simplified, as again empirical knowledge can be used to perform feature selection.

Note that this does not mean that we are trying to solve a very confined problem. Indeed, the goal here is to be able to perform analysis and prediction of any metric and any process. Still, the fact that we are dealing with business processes makes the problem practically tractable, as discussed in the following.

Transformer. To learn prediction models, the BPI engine needs to retrieve the data about the process instances on which metrics are computed. However, this data is often spread among different tables (see the fact tables in figure 4) and since data mining algorithms require all data to reside in one table with one fixed length record per training instance,

the execution data retrieved from the data warehouse needs to be transformed into such a format. Note that this is not only a technical problem: the real challenge here is defining the structure of the table to be fed to the mining algorithms, since this will determine the features considered for performing the analysis and prediction.

In particular, the problem is represented by data structures that have 1 to N relationship with the process instance (for example, a process instance includes several activities), where N is possibly unbounded. In fact, for such data, there are several records per each process instance. For example, an order process instance may need more than one approval submission, hence there will be more than one record for each of those activity executions. Each such record will include data about a different approval. Since, as stated above, only one fixed length record per training instance is accepted by the data mining algorithms, a transformation has to take place on the n records associated to a process instance, to be able to “compress” them into a single record. Such transformation requires making a choice about which of the n records will be included in the training table. This is an example of where empirical knowledge about process mining and knowledge about mining of generic metrics come into play. For example, according to previous experiments, we have witnessed that the first and last execution of a cycle are the ones that have more influence on the overall *duration* of a process instance. This knowledge is embodied into the Transformer. For user defined metrics whose semantics is unknown a priori to the BPI engine, the burden of the task could be left to the user, but it is also possible to use default settings that are proven to work for most metrics. For example, the strategy of taking the first and last iterations of loops seem to work in general. Other examples of parameters that we have empirically found to be generally applicable are the number of times a node has been executed in the instance or the average duration of a node that appears in a loop. These statistics (along with information on the first and last execution of the node) provide a fixed amount of information about an unbounded number of node executions. We will return later on to the issue of the specific data being collected about a node. At this time, the goal of the explanation is to stress that experience can be applied to identify a finite set of features that can be collected, in a way that is independent on the process and the metric being analyzed, thereby enabling the automation of this procedure in the general case.

The transformation of process instances into training instances presents another complication. As explained in Section 4.5, different prediction models need to be created for the different stages that process instances go through their execution. Different stages differ from each other in the last node reached and/or the path followed to reach that stage. This implies that not only the stages that have occurred during past process executions have to be identified from the data, but also that different transformations are necessary to only consider the data that exists at each one of these stages. The BPI engine automatically identifies the various execution stages and the Transformer makes all the necessary transformations to prepare the data (training set) for these stages.

Another typical case of transformation is on time series, introduced in Section 4.5. A series needs to be transformed into a standard case representation where the window (or lag time) size is critical for learning a good predictive model. To illustrate the point, let us assume we have a time series S of the daily average duration of process instances during

one month: $S = 8.5, 5.3, 7.9, 9.5, 6.4, 10.3, \dots, 5.2, 6.8, 7.2, 8.7$. The setting of the window size of the cases derived from this time series is not trivial at all, plus it has to be automatic. For this effect, BPI engine uses the autocorrelation function of the time series to guide this setting [5]. In addition, the original data is not always the best to use, sometimes models are more accurate when new attributes are composed from the original ones. Examples of composition functions are the average of the values in a window, accumulated values, a ratio to indicate the percentage change, or a difference to indicate a net change. The challenge is to identify which of these is the most suitable for each time series. Our solution is to let the Feature Generator (below) derive some of the most typical compositions, then have the Transformer convert the cases accordingly and finally let the Feature Selector identify the most relevant one.

Another transformation which is simple but important is the one occurring after the most relevant features have been identified (see Feature Selector below). In this case, the training set is projected into the features (attributes) selected, discarding in this way the data about irrelevant features.

Feature selector. The feature selection problem [15, 22] is the process of selecting a relevant subset of features upon which a data mining algorithm will focus its attention. The inclusion of irrelevant, redundant and noisy attributes can result in poor predictive performance and increased computation time. According to the kind of knowledge used to assess the relevance of attributes we distinguish two classes of feature selection: a *semantic* one and a *mechanical* one. Semantic feature selection is mostly used when we want to learn models for explanation purposes. In this case, domain knowledge is used to determine whether an attribute, according to its meaning, would be uninteresting if it appeared as part of the explanation. For generic metrics, this knowledge is encoded in the BPI engine. Mechanical feature selection does not instead consider the semantics of attributes. There are two basic approaches for this kind of feature selection. In the first one, called *filter*, general characteristics of the data are used to evaluate the relevance of each individual attribute with respect to the target concept according to some metric, like chi-square or information gain. In the second approach to feature selection, different subsets of the original attributes are evaluated with respect to the performance of the models obtained and the subset that yields to the model with the best performance is identified. Since the feature selection wraps the learning of the model and its evaluation, this approach is called *wrapper* feature selection. Several works in the 90s have shown the superiority of the approach, in terms of predictive accuracy over the filter one. Therefore, BPI engine uses the wrapper approach for feature selection whenever the size of the problem allows its application. As an example, the training set to learn a prediction model for metric *cost* of the supply chain process includes not only data that any process instance has like *start time* and *duration*, but also data about its parameters like *customerID*, *productID*, *quantity* and *requestedDeliveryDate*. After feature selection, only some of them like *duration*, *productID* and *requestedDeliveryDate* were found as the subset of relevant features. Once the features have been selected, the feature selector not only sends them to the Transformer, but also to the Process Data Store for later retrieval at prediction time.

The key problem in the context of the process management domain is the identification of the initial features to be fed to the feature selection algorithms. Again, there are many different possible features and, even when the number of node executions is bounded, the number of features is not. For example, one could assume that the time difference between the activation timestamps of two certain nodes in the process or the ratio between their two durations are relevant features. We found the following features to be generally useful, and therefore to be candidate for feature selection:

- node activation and termination timestamps, decomposed into month, day of the week and of the month, and hour of the day
- process activation and termination timestamp, decomposed as above
- number of executions for each node
- duration for each node
- value of the process variables

Feature generator. When we described the Transformer we mentioned that in time series new attributes composed from the original ones often permit the derivation of more accurate prediction models. The Feature Generator is responsible of the generation of these new attributes. A limited set of composition functions which have shown good results in previous experiments are embodied in this module to compute new attributes for the cases of a time series. Also, the decomposition of the timestamps of the node and process instances, the computation of the number of executions of a node and its duration take place in this module. Once all these features have been computed upon the Transformer request, the Feature Generator passes them back to it to use them in the creation of the training set. The final attributes of the training set, will be chosen by the Feature Selector.

Attribute descriptor generator. Data mining algorithms require a description of each of the attributes of the training set in a specific format. As a common denominator, the data type and the position of each attribute need to be specified along with an enumeration of the classes when a nominal label attribute exists. The approach used by the Attribute Descriptor Generator is to generate attribute descriptor files in a canonical format and map these files into the specific format required by the data mining algorithm (a conversion method for each specific format exists).

5.1.2. Data mining modules. Once all the data (i.e. training set and attribute descriptors) required to learn a prediction model for a given metric have been retrieved and prepared, they are input to the *Model Generator* module to forgo all the steps required to learn a good model. This task implies three main challenges:

- identifying suitable techniques for each class of problems.
- automatically tune each applicable technique for the best possible performance for the problem at hand.
- automatically identify which of the techniques, already tuned for their best performance, performs best.

As we will see, the Model Generator implements a solution to each of these three challenges into its various submodules: model selector, model learners, parameter optimizer and validator.

Model selector. This module is an implementation of the results obtained in response to the first challenge, summarized in Table 1 (Section 4). As Table 1 indicates, the characteristics of each class of problems determine which data mining techniques are suitable to solve problems in that class. Given a request for metric explanation or prediction, the Model Selector analyzes the kind of task (prediction vs. analysis), metric scope (generic vs. user-defined), focus of the prediction (targeted vs. untargeted) and status of the instances subject to the prediction (active vs. future) to automatically determine which data mining algorithm to apply. The knowledge embedded in this module is the result of an experimental evaluation of the techniques on different business processes. Thus, the user doesn't need to conduct experiments to assess which technique to apply for each particular problem. For example, to learn a model that explains when the cost of processing an order is 'high', the Model Selector identifies that the task is explanation, the metric scope is user-defined (its semantics is specific to supply chain processes) and the focus is targeted (the explanation is for the cost metric). Therefore, it selects decision trees as the candidate technique and sends this information to the Parameter Optimizer (see below) to activate the generation of parameter values for the corresponding learner.

Model learners. A set of learners, one for each data mining technique in Table 1, is included in our platform. A model learner receives as input the training set prepared by the Transformer along with the attribute descriptions prepared by the Attribute Descriptor Generator, and applies its algorithm to learn a model from this data set. A learner is first applied within an evaluation process where its performance is evaluated (see Validator below). After being validated, the learner is applied again but this time on the complete training set to learn the final model. The output is the model itself, which in the first case is sent to the Model Applier to be applied on a test set and in the second case it is loaded in the Process Data Store. For our example of explanation on the cost of processing orders, the decision tree learner selected by the Model Selector is the one being activated. The set of learners in the BPI engine is not intended to be exhaustive, however it is easily extendible.

Parameter optimizer. A Model Learner has a set of parameters that need to be tuned for the problem at hand to obtain the best performance out of it. The Parameter Optimizer is in charge of producing different combinations of values for the different parameters of each technique and sending them to the corresponding Model Learner. BPC uses generic optimizers, that is, there is no process-specific aspect in this component.

Validator. Once a model is learned, its performance needs to be evaluated. There are different methods to do this validation among which cross validation is the preferred one. The idea is to split the training set into different subsets that are iteratively grouped in different ways, so that in each iteration all the subsets, except one, are used for learning a model, while the subset left out during the learning phase is used for testing the model. The partitioning of the training set takes place before any model is learned. Once a model is

learned on the portion of the training set corresponding to the actual iteration, the validator invokes the Model Applier on the left out subset to test the model. Different metrics can be used to measure the performance depending on the task. For example, for measuring the performance of a decision tree learned for the order process cost metric, accuracy, precision and classification error are good criteria to use. Again, we use generic validation techniques here, as the problem is generic and not specific to process analysis. Hence, we do not discuss this further.

5.2. *User interaction*

Users can access explanations and predictions through the GUI of the reporting engine. Whenever explanations or prediction models have been derived for a report, the GUI will display corresponding *explanation* and *prediction* buttons next to the chart. If users request an explanation, then the explanation model is retrieved and presented to the user either graphically or in textual form (examples are provided later in the paper). If users request a prediction on an active process instance, then the prediction model is applied to the instance data and the predicted value is displayed.

For reports based on metrics grouped by time, the panel with the report of the metric on historical data incorporates a “time slider” that lets the user select the time for the prediction. For example, for metric “average number of delayed order transactions” grouped by month, the time slider lets the user select the month for which the predicted value of the metric is to be computed. This is illustrated in figure 9, where the predicted values are shown on the charts.

6. **Experimental results**

We have applied the concepts, techniques and framework discussed in the previous sections to the intelligent analysis of several processes. In particular, we have conducted some experiments with a preliminary version of the Business Process Cockpit on the supply chain process presented in Section 2. In this section, we report some interesting findings from those experiments as an illustration of the value that can be obtained from using a tool that automates the intelligent analysis and prediction of business processes.

We had access to 4 months of supply chain process execution data. We defined metrics on this application, in particular, metric *outcome* which takes value “accepted” if activity *notify_acceptance* was executed, and “rejected” if instead, activity *notify_rejection* had taken place in a supply chain process instance. Once defined, the cockpit computes the metrics, which means that process instances became labeled with metrics values. The reporting engine of the cockpit can then display charts (such as the bar chart in figure 8) that give information about the number and proportions of instances labeled with each of the different metric values.

Noticing that a big proportion of process instances had been labeled as “rejected”, the user can exploit the intelligent component of the cockpit to request an explanation for why so many orders could not be fulfilled (he or she will do so by following a link available from the page where the report is displayed).

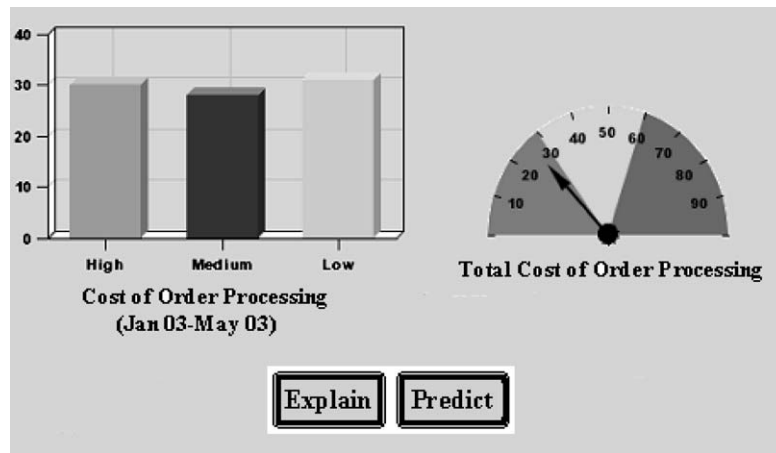


Figure 8. Report panel with clickable buttons.

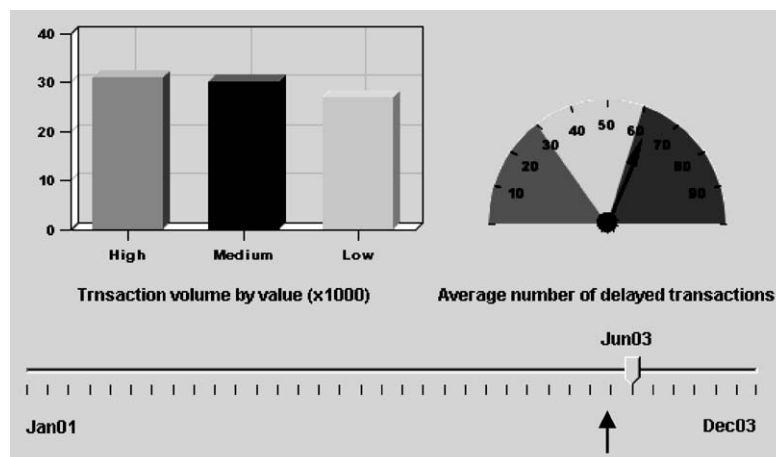


Figure 9. Prediction panel with time slider (arrow on slider shows May 03 as current date).

For our case study in question, since metric outcome was defined on the supply chain process, the historical execution data of its instances (see figure 4), including data about the orders (i.e. customer, product, requestedDeliveryDate and quantity), were retrieved from the Process Data Warehouse and passed to the BPI engine to be transformed and mined for an explanation and prediction model. Since the metric to be analyzed takes nominal values, the explanation model can be a decision tree or decision rules, both are generated. The steps conducted to generate these models have been explained in Section 5, and are not repeated here. For the sake of simplicity and confidentiality, we do not show the exact decision tree and some details of the results, but in figure 10, we show a partial, simplified version of the tree that captures interesting results. In fact, a simplified tree is the one that we aim to

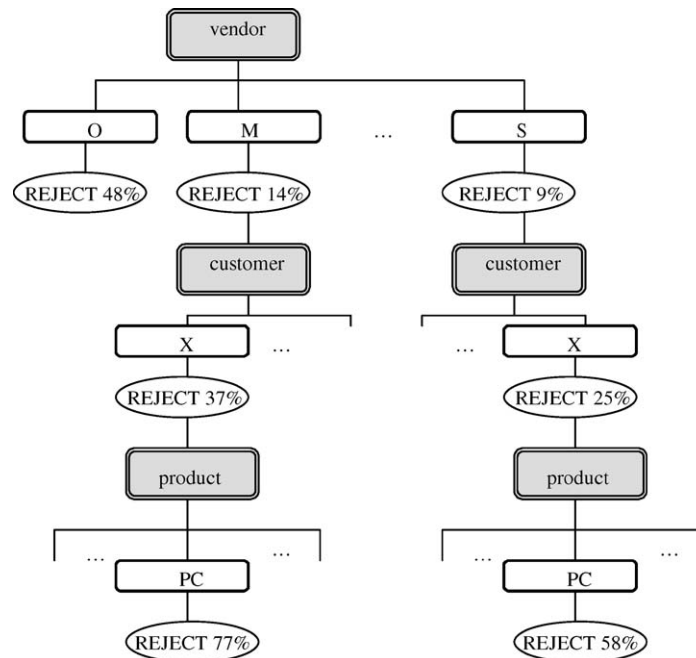


Figure 10. Simplified decision tree for outcome metric.

show up when an explanation for a metric is requested on line. Original versions of trees are often large, and therefore not easily interpretable by the end users. We are working on the post-processing of trees to automatically identify its interesting parts and transform them into a simple representation that conveys valuable information. Notice that models are built off line so that when explanations or predictions are requested on line, the response time is minimal.

The tree shown in figure 10 shows some very interesting findings: we notice that 48% of the times that the supplier does not have enough supplies in stock and orders them from vendor O, the orders cannot be fulfilled. This is an indication that vendor O often does not have enough supplies or is not efficient enough in shipping goods to promise that the shipment will arrive on time. Since the goal of the supplier is to maximize the fulfillment of orders, this valuable information can be used to take corrective action. For example, this can consist in searching for another vendor, who can consistently deliver goods in the time allotted, to replace vendor O.

The next question that follows is: can we predict the outcome of an order (supply chain process instance) with the explanation model learned? The answer is yes and no. As the decision tree in figure 10 shows, the first node splits according to the value of the vendor attribute, but as shown in figure 2, a vendor is only required after activity `verify_supplies_in_stock` has been executed and it has been found that there are not enough supplies to fulfill the order. Therefore, only until this moment the decision tree can be used to predict the outcome.

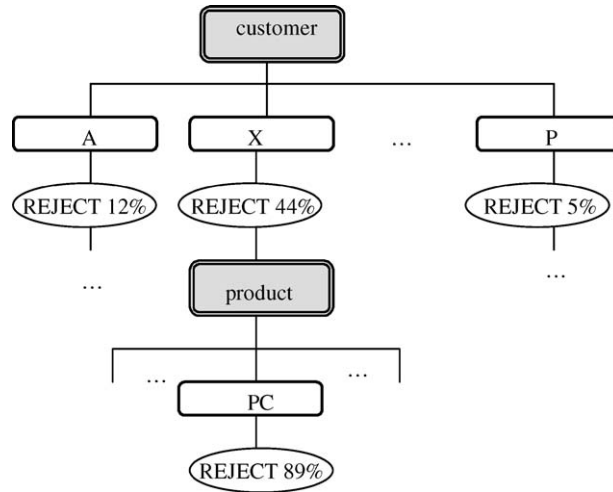


Figure 11. Partial decision tree for predicting outcome at initial stage.

Nevertheless, the prediction gives the supplier the opportunity to be proactive, when the prediction is for “reject” with a high probability, instead of just wait for the vendor’s reply. However, since predictions are needed as early as possible, other prediction models are built for earlier process instance stages. Of particular interest is the prediction model for the initial stage because it has the power to make predictions when the order has just been received. The decision tree is simple because only data about the order itself is known at this initial stage. Nevertheless, as can be observed in figure 11, the tree is useful enough to predict that when the order has been placed by customer *X* on product *PC*, with 89% probability, the order will be rejected. In this case, the supplier can also be proactive and immediately act to prevent the rejection. Prediction models give him the capability to detect problems before they happen.

Finally, we briefly describe another useful application of prediction for our example case. Again, in order to maximize the fulfillment of orders, the supplier not only needs to understand the reasons why orders are not fulfilled, or have predictions of an order final outcome early during its processing, but he would also like to have predictions of the quantity of products that will be ordered in a future time so that he can adjust the stock levels accordingly and minimize the dependency on other vendors at order processing time. To respond to this need, prediction models are obtained for various time series depending on the time window that the supplier is interested in. Typically, the analyst can obtain reports for the number of products sold during a week, a month and a quarter, or other time units. Consequently, it is likely that he would like to know the prediction for next week, next month and next quarter. A time series for each time unit (e.g., week, month, and quarter) is produced from the `quantity` metric values in the Process Data Store. For instance, the weekly time series is a sequence of values corresponding to the quantity of products sold during successive weeks: `qty-week1`, `qty-week2`, `qty-week3`, ..., `qty-weekn`. The desired

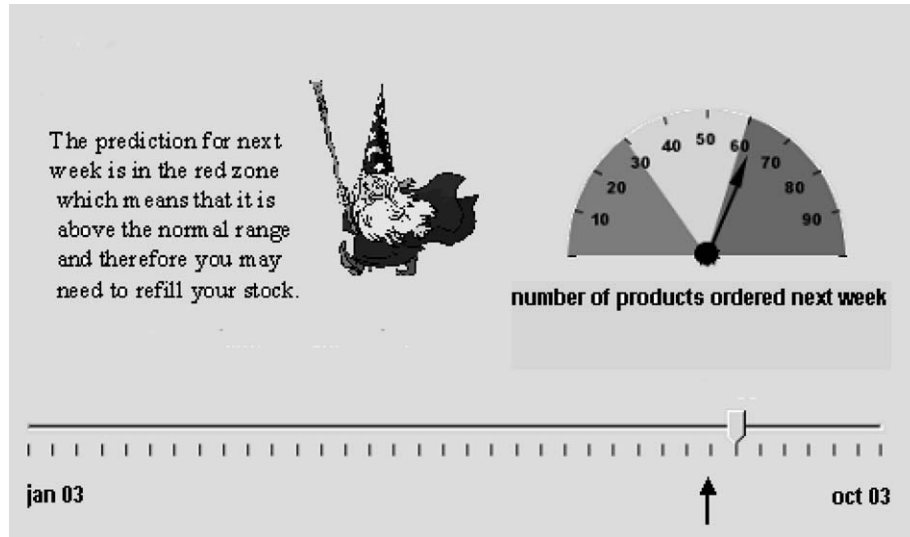


Figure 12. Prediction for metric quantity of products per week for next week.

prediction is for the quantity for week $n + m$, where m usually takes small values given that the farther the prediction is in the future, the lower is its accuracy.

Having the prediction for next week, as shown in figure 12, the supplier can assess whether the level of its stock is adequate or not. If the level is not enough to fulfill the quantity of products predicted, the supplier can reorder goods from its vendors much in advance. This intelligent approach to refill stocks is more effective than its simplistic threshold-based counterpart because it considers the dynamics of time. Thresholds are often not enough because they do not take into account the load patterns exhibited in the past. In figure 12 we also show another capability that we would like to fully incorporate into the BPI engine, that of autonomously detecting abnormal values for a prediction and sending alerts accompanied with suggestions. This functionality could be realized with a rule engine instead of coding specific applications that are executed after a prediction is computed.

This example has illustrated the value provided by an intelligent approach to the analysis and prediction of business processes. As we have seen, explanations and predictions are essential for the supplier to achieve his goal of maximizing the fulfillment of orders by giving him the opportunity to understand problems, detect them before they happen and proactively react.

7. Related work

In this Section we examine prior art related to different aspects of the work presented in this paper. Several research projects deal with the technical facilities necessary for the logging of the audit trail data. In this paper we do not deal with this aspect of the problem. This was

the subject of another paper [6] where the data warehouse for the Business Process Cockpit is fully described. In this respect, there are other papers like [12, 23, 26] which also discuss the design of a data warehouse for workflow logs. zur Muehlen [25] develops a taxonomy of different analysis purposes for workflow audit data. This taxonomy helps in determining the relevant data for analysis and their aggregation. They distinguish between analysis of active workflows and post-analysis of completed instances. Other criteria of differentiation are the purpose of the analysis (for technical or business-oriented analysis goals) and the type of user (individual user or organization). Eder et al. [12] proposes a data warehouse hypercube with six dimensions: workflows, participants, organization, servers, time and measures. List et al. [23] describes the process warehouse as an enabler for process analysts to receive information on business processes very quickly, at various aggregation levels, from different and multidimensional points of view, over a long period of time, using a huge historical data basis. In [13], the authors present an approach based on active databases for the logging and post-mortem analysis of workflow executions. Database transitions, time events and external events in the history can trigger the evaluation of a condition that when satisfied activates the execution of an action. The objectives in all these works are different than ours. In fact, our initial focus with the cockpit focused on analysis of business metrics, not just reporting of execution data. Furthermore, and most importantly, in this paper we discuss the automated application of sophisticated data mining techniques to analyze and predict any business metric, as opposed to using a certain statistical approach to solve a single, specific problem.

Other works have addressed the monitoring of workflows. The original Business Process Cockpit addressed this aspect and is documented in [32], but as we have seen it has evolved from a passive process data warehousing platform to a business process intelligence solution. zur Muehlen and Rosemann [25] focuses on the analysis of the logged audit trail data, and proposes a tool (called PISA) that integrates the analysis of the audit trail data with the target data obtained from business process modeling tools.

Several commercial products (Tibco, Categoric, Actimize) offer solutions for business activity monitoring (BAM). BAM is a term coined by Gartner Group that refers to analysis and presentation of relevant and timely information about business activities across multiple applications, data formats, and systems. BAM is still an emerging area and addresses only one piece of the overall BPI challenges, providing a view of historical performance through monitoring business processes. We view the “M” from BAM as “management”, which means that we go further than simple monitoring, we provide intelligent analysis and prediction of business activities. On the other hand, several BI and analytic application vendors claim to offer the ability to optimize business processes, but their strengths in reporting and analysis have yet to be aligned in a process context. Vendors like Oracle, Cognos Inc., Hyperion Solutions Corp., and Informatica Corp. have added workflow support to enable event and notification-based support, but they focus merely on adding process metrics to their product architectures for traditional reporting and analysis. In contrast, we propose an integrated approach that supports explanations and predictions of processes behavior from a business perspective.

Other works like [2, 8, 18, 19] have used data mining to automatically derive a formal model of a process from a log of events related to the executions of a process that is not

supported by a process management system. As we mentioned in Section 3, the Business Process Cockpit does not deal with this particular task, so we won't discuss these works, a detailed survey of this research area is provided in [1].

To the best of our knowledge, there are no approaches to process execution analysis and prediction based on data warehousing and data mining techniques for Business Process Intelligence. A few contributions exist in the field of exception prediction, however limited to estimating deadline expirations. In [29] and [30], the authors address the problem of time management, i.e. of predicting as early as possible when a process instance is not likely to meet its deadline. In [29] every activity in the process has an assigned maximum duration. When the maximum duration is exceeded, the process is escalated. When an activity executes faster than its maximum duration, a slack time becomes available that can be used to dynamically adjust the maximum durations of the subsequent activity. In [30] the activity completion time is estimated based on the current load of the system resources and on the average execution time of the activity, calculated on past executions. Estimated activity execution times are then used to estimate the duration of the process, and to decide whether the process should be escalated. In [11] the duration of an activity can be defined by the designer or determined based on past executions. In addition, the designer may define deadlines (latest allowed completion time relative to the process start time) for activities or for the whole process. Process definitions are translated into a PERT diagram that shows for each activity, based on the expected activity durations and on the defined deadlines, the earliest point in time when the activity can finish as well as the latest point in time when it must finish to satisfy the deadline constraints. The PERT technique is extended to handle process definitions that include alternative and optional activities. During the execution of a process instance, given the current time instant, the expected duration of an activity, and the calculated latest end time, the progress of the process instance can be assessed with respect to its deadline. This information can be used to alert process administrators about risk of missing deadlines and to inform users about the urgency of their activities. In contrast, we aim at predicting any kind of exception, rather than focusing on deadline expirations. In addition, we propose to build explanation and prediction models by leveraging data warehousing and data mining techniques based on characteristics of the process instances, such as data values, resources, the day of the week or hour of the day in which processes or activities are started.

As we can observe, our approach is unique in that we aim to cover the whole management aspect of business processes, i.e. monitoring, analysis, prediction, control and optimization in an intelligent and integrated way. We include in our framework all the necessary infrastructure, techniques, and concepts necessary to achieve this goal, as well as an architecture as the realization of the framework.

8. Conclusions and future work

This paper has presented a set of concepts, techniques, and architectures that enable the automated application of data mining techniques to business process execution, to both analyze and predict metrics of interest to business and IT analysts. The main contribution of

this paper consists in presenting an approach that makes “intelligent” functionality available to the user with a fraction of the effort (and at a fraction of the cost) normally required when applying such sophisticated techniques. The price to be paid is a reduced accuracy in the analysis and prediction models with respect to what can be achieved by a long and focused data mining project, but it is our experience that customers are extremely interested in the immediate, low cost availability of intelligent analysis that they are more than willing to pay this price.

Our current work aims at refining and extending this approach in several directions. One of them involves the application of *incremental* and *adaptive* data mining algorithms [17, 24, 31], so that the model can be updated without having to be recomputed from scratch. This is important in those applicative domains where the environment changes often, and therefore the model may quickly become obsolete if not updated frequently. Another research direction involves identifying ways to present analysis results in a way that is easy to digest, even by users that are not expert in data mining. Finally, we plan to extend the approach to handle other classes of middleware platforms, going beyond workflows.

Note

1. A case corresponds to the current and previous values at each moment in time.

References

1. W.M.P. van der Aalst, B.F. van Dongen, J. Herbst, L. Maruster, G. Schimm, and A.J.M.M. Weijters, “Workflow mining: A survey of issues and approaches,” internal report, <http://tmitwww.tm.tue.nl/staff/wvdaalst/workflow/mining/wf-min-surv.pdf>.
2. R. Agrawal, D. Gunopulos, and F. Leymann, “Mining process models from workflow logs,” in Proceedings of the 6th International Conference on Extending Database Technology (EDBT), Valencia, Spain, 1998.
3. G. Alonso, F. Casati, H. Kuno, and V. Machiraju, *Web Services: Concepts, Architectures, and Applications*, Springer Verlag, 2003.
4. L. Breiman, *Classification and Regression Trees*, CRC Press, 1984.
5. P. Brockwell, *Introduction to Time Series and Forecasting*, 2nd edition, Springer-Verlag, March 2002.
6. F. Casati, “Intelligent process data warehouse for HPPM 5.0,” HP Labs technical report HPL-2002-120, 2002, Available from www.hpl.hp.com.
7. M. Castellanos, F. Casati, U. Dayal, and M. Shan, “Intelligent management of SLAs for composite Web services,” Third International Workshop on Databases in Networked Information Systems, Springer Verlag, 2003.
8. J. Cook and A. Wolf, “Discovering models of software processes from event-based data,” *ACM Transactions on Software Engineering and Methodology*, vol. 7, no. 3, 1998.
9. N. Cristianini and J. Shawe-Taylor, *An Introduction to Support Vector Machines*, Cambridge University Press, 2000.
10. R.O. Duda, P.E. Hart, and D.D.G. Stork, *Unsupervised Learning and Clustering*, 2nd edn., Chapter 10, *Pattern Classification*, Wiley InterScience, 2001.
11. J. Eder, E. Panagos, H. Pozewaunig, and M. Rabinovich, “Time management in workflow systems,” in Proceedings of the BIS’99, Poznan, Poland, 1999.
12. J. Eder, G. Olivotto, and Wolfgang Grube, “A data warehouse for workflow logs,” in Proceedings of the International Conference on Engineering and Deployment of Cooperative Information Systems (EDCIS 2002),

- Beijing, China, Sept. 17–20, Springer Verlag (LNCS 2480), 2002, pp. 1–15, ISBN 3-540-44222-7, ISSN 0302-9743.
13. A. Geppert and D. Tombros, “Logging and post-mortem analysis of workflow executions based on event histories,” in *Proceedings of the 3rd Intl. Conf. on Rules in Database Systems (RIDS)*, LNCS 1312, Springer, Sweden, June 1997.
 14. D. Grigori, F. Casati, U. Dayal, and M.C. Shan, “Improving business process quality through exception understanding, prediction, and prevention,” in *Procs. of VLDB’01*, Rome, Italy, Sept. 2001.
 15. M. Hall and G. Holmes, “Benchmarking attribute selection techniques for data mining,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 15, no. 3, 2003.
 16. J. Han and M. Kamber, *Data Mining Concepts and Techniques*, Academic Press: Morgan Kauffmann Publishers, 2001.
 17. G. Hulten, L. Spencer, and P. Domingos, “Mining time changing data streams,” in *Proceedings of the Seventh ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, ACM Press, 2001.
 18. J. Herbst, “An inductive approach to adaptive workflow systems,” *CSCW-98 Workshop, Towards Adaptive Workflow Systems*, Seattle, WA, USA, 1998.
 19. J. Herbst and D. Karagiannis, “Integrating machine learning and workflow management to support acquisition and adaptation of workflow models,” in *Proceedings of the Ninth International Workshop on Database and Expert Systems Applications*, IEEE, 1998, pp. 745–752.
 20. G.H. John, R. Kohavi, and K. Pfleger, “Irrelevant features and the subset selection problem,” in *Proc. of the 11th International Conference on Machine Learning ICML94*, 1994.
 21. F. Leymann and D. Roller, *Production Workflow*, Prentice-Hall, 2000.
 22. H. Liu and H. Motoda, *Feature Selection for Knowledge Discovery and Data Mining*, Kluwer Academic Press, 1998.
 23. B. List, J. Schiefer, A.M. Tjoa, and G. Quirchmayr, “Multidimensional business process analysis with the process warehouse,” in *Knowledge Discovery for Business Information Systems*, Chapter 9, Kluwer Academic Publishers: Boston, 2000.
 24. R. Klinkenberg and T. Joachims, “Detecting concept drift with support vector machines,” in *Proceedings of the Seventeenth International Conference on Machine Learning (IVML)*, San Francisco, 2000.
 25. M. zur Muehlen and M. Rosemann, “Workflow-based process monitoring and controlling—Technical and organizational issues,” in *Proceedings of the 33rd Hawaii’ International Conference on Systems Sciences*, Wailea, HI, 2000.
 26. M. zur Muehlen, “Process-driven management information systems—Combining data warehouses and workflow technology,” in *Proceedings of the 4th International Conference on Electronic Commerce Research (ICECR-4)*, B. Gavin (Ed.), Dallas, TX, 2001, pp. 550–566.
 27. Michael zur Muehlen, *Workflow-Based Process Controlling—Or: What You Can Measure You Can Control*, *Workflow Handbook*, Layna Fischer, 2001, *Future Strategies*, Lighthouse Point, FL 2001, pp. 61–77.
 28. P. Muth, J. Weissenfels, M. Gillmann, and G. Weikum, “Workflow history management in virtual enterprises using a light-weight workflow management system,” in *Proceedings of the 9th International Workshop on Research Issues in Data Engineering*, Sydney, Australia, 1999, pp. 148–155.
 29. E. Panagos and M. Rabinovich, “Escalations in workflow management systems,” in *Proceedings of the DART’96*, Rockville, MD, 1996.
 30. E. Panagos and M. Rabinovich, “Predictive workflow management,” in *Proceedings of the NGITS’97*, Neve Ilan, Israel, 1997.
 31. S. Ruping, “Incremental learning with support vector machines,” in *Proceedings of the 2001 IEEE International Conference on Data Mining*, 2001.
 32. M. Sayal, F. Casati, U. Dayal, and M.C. Shan, “Business process cockpit,” in *Procs of VLDB’02*, Hong Kong, China, Aug. 2002.
 33. H. Schuster, D. Baker, A. Cichocki, D. Georgakopoulos, and M. Rusinkiewicz, “The collaboration management infrastructure,” *ICDE*, pp. 677–678, 2000.